¿Cuál variante nos conviene paralelizar?

Hasta ahora habíamos elegido right-looking por su organización natural de las actualizaciones, pero eso era una razón de diseño. Para responder con evidencia implementamos también left-looking con MPI y comparamos ambas variantes bajo las mismas condiciones.

La dependencia que comparten

En ambas variantes, la columna j solo puede finalizar cuando están disponibles las contribuciones que la afectan. Right-looking no elimina esa dependencia ni permite calcular arbitrariamente todas las columnas a la vez.

Dónde aparece el trabajo paralelo

Variante	Antes de calcular j	Trabajo repartido después
Left-looking	Reúne desde las columnas 0,...,j-1 las contribuciones necesarias para j.	Con el pivote listo, calcula en paralelo distintos elementos Lij de la columna actual.
Right-looking	Las columnas anteriores ya aplicaron sus aportes sobre la matriz de trabajo.	Con j lista, actualiza en paralelo muchas posiciones de las columnas futuras.

$$l_{ij} = \frac{a_{ij} - \sum_{k<j} l_{ik} l_{jk}}{l_{jj}} \quad \text{(left)}$$

$$s_{ik} \leftarrow s_{ik} - l_{ij} l_{kj}, \quad i \geq k > j \quad \text{(right)}$$

Una formulación precisa

Left-looking reúne el pasado para construir la columna actual. Right-looking propaga la columna actual hacia el futuro. Ambos admiten paralelismo; right-looking expone un conjunto amplio de actualizaciones sobre la región restante. En versiones por tareas se puede priorizar la próxima columna y continuar otras actualizaciones como look-ahead.

Nuestra versión right-looking es sincronizada por columnas: difunde j y cada proceso actualiza sus columnas propias antes de la etapa siguiente. Por eso esta comparación evalúa dos implementaciones concretas, no el máximo paralelismo teórico posible de cada familia.

Cómo hicimos comparable el experimento

La misma propiedad de datos

Ambas variantes usan distribución cíclica estática. Cada proceso almacena sus columnas en memoria local contigua y recibe exactamente la misma matriz SPD. La distribución inicial y la reunión final son idénticas y quedan fuera del cronómetro del kernel.

Left-looking MPI	Right-looking MPI
Cada proceso calcula sumas parciales con sus columnas anteriores.	El propietario calcula la columna j.
MPI_Reduce reúne el pivote y los elementos inferiores en el propietario de j.	MPI_Bcast difunde el tramo terminado de j.
El propietario guarda la nueva columna; se difunde el pivote.	Cada proceso actualiza sus columnas futuras con vector local contiguo.

Diferencia de coordinación

Por columna, left usa una reducción escalar, una reducción vectorial -salvo en la última columna- y una difusión escalar: hasta 3n-1 colectivas. Right usa una difusión vectorial por columna: n colectivas. Los volúmenes exactos dependen de la implementación de MPI, pero left introduce más puntos de coordinación.

Protocolo

Decisión	Valor
Tamaños y procesos	n=64,128,256,512,1024; P=1,2,3,4.
Repeticiones	Un calentamiento y ocho mediciones por algoritmo y configuración. Orden left/right alternado.
Entorno	WSL2, AMD Ryzen 5 2600, Open MPI 5.0.10, GCC 15.2.0, -O3, double, sin BLAS.
Tiempo	Máximo de las duraciones locales tras una barrera. Incluye cálculo y colectivas; excluye Scatterv, Gatherv y validación.
Correctitud	Residuo relativo <=10^-12 y comparación con el factor right secuencial.

También ejecutamos un perfil separado para n=1024 y P=3. Sus cronómetros internos alteran el tiempo total; se usan para interpretar componentes, no para sustituir las medianas principales.

Resultado: right fue más rápido en todos los casos

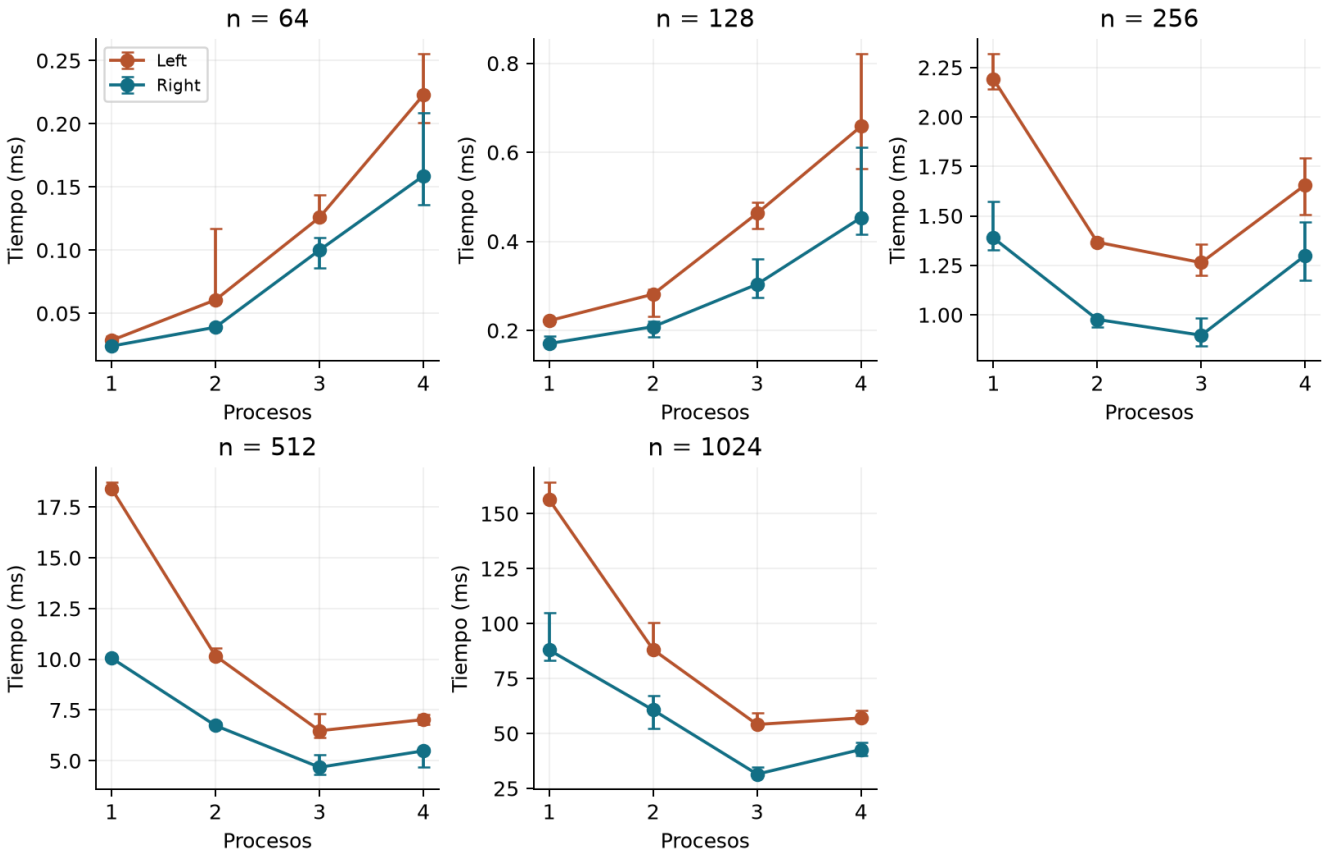


Figura 1. Medianas de ocho repeticiones; barras Q1-Q3. Se usa una escala propia por panel.

n	P	Left (ms)	Right (ms)	Left / Right
64	1	0,03	0,02	1,19
128	1	0,22	0,17	1,30
256	3	1,26	0,90	1,41
512	3	6,48	4,68	1,38
1024	3	54,08	31,48	1,72

La tabla usa, para cada n, el P que minimizó la mediana de right. No selecciona P por separado para favorecer a left. En las 20 configuraciones completas, right tuvo menor mediana.

Caso principal: n=1024

Con tres procesos, left tardó 54,08 ms y right 31,48 ms: left fue 1,72 veces más lento. Respecto de su propio P=1, left alcanzó 2,89x y right 2,79x. Ambos mejoraron al paralelizar y ambos empeoraron al pasar de tres a cuatro procesos.

Por qué right ganó en nuestra implementación

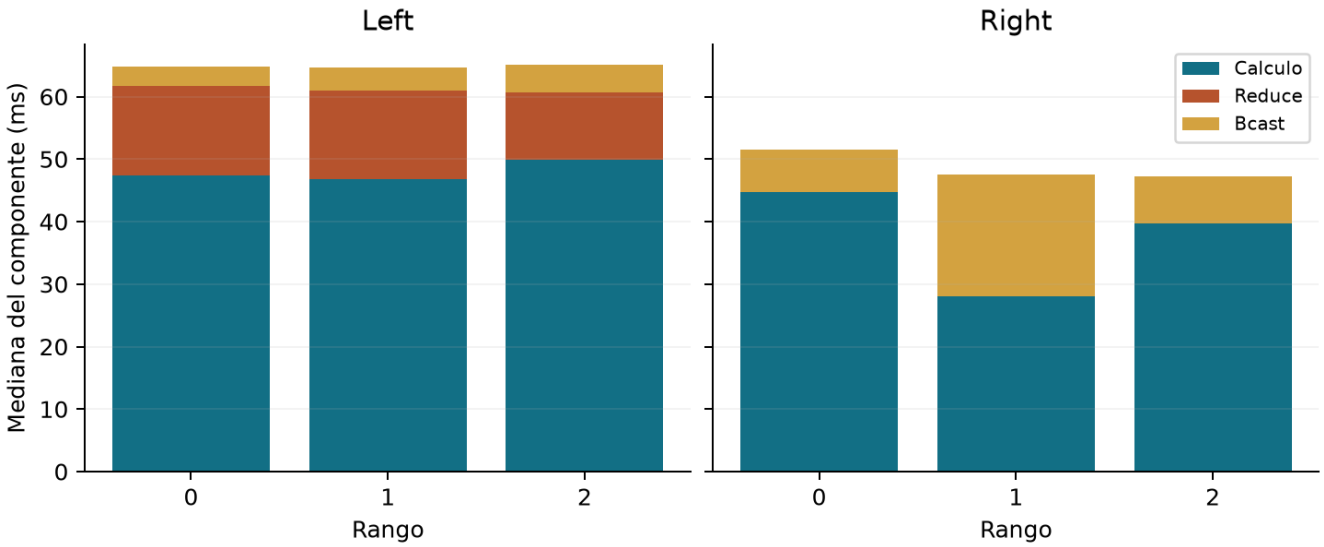


Figura 2. Perfil separado de n=1024, P=3, cuatro repeticiones por algoritmo. Se apilan medianas de componentes; su suma no tiene por qué coincidir exactamente con la mediana del total. Las colectivas incluyen espera.

La comunicación de left es más fragmentada

Left necesita reunir sumas parciales en cada columna. Aunque reparte bien los productos locales, paga reducciones vectoriales y escalares además de difundir cada pivote. Right difunde la columna terminada una vez y realiza actualizaciones locales sobre los datos que cada proceso posee.

En el perfil, los tres rangos de left acumulan aproximadamente entre 10,80 y 14,29 ms dentro de MPI_Reduce, además de sus difusiones. Right no usa reducciones en el kernel. Esta observación explica parte de la diferencia, pero el tiempo dentro de una colectiva mezcla transferencia y espera.

No es una prueba universal

La conclusión válida es: **para nuestra implementación escalar, almacenamiento por filas, distribución cíclica, Open MPI local y tamaños probados, conviene continuar paralelizando right-looking con vector local.** No demostramos que toda implementación right sea superior a toda left.

Una versión left por bloques, con BLAS, menos reducciones o comunicación no bloqueante podría cambiar el resultado. Una versión right con look-ahead también podría mejorar. Esas comparaciones requieren otra etapa y no deben darse por realizadas.

Validación

Se aceptaron 320 tiempos del experimento y 24 registros de perfil. El residuo máximo fue 1,2131×10^-15. La máxima diferencia relativa elemento a elemento frente al factor secuencial fue 2,6578×10^-15. Las pruebas con P=1,2,3,4 y los sanitizadores con P=4 finalizaron correctamente.

Decisión y explicación para la consulta

Qué elegimos

Continuamos con right-looking MPI cíclico y vector local. La elección tiene ahora dos fundamentos: su estructura permite repartir las actualizaciones futuras entre propietarios de columnas, y la comparación directa mostró menor tiempo que el left-looking MPI implementado con el mismo reparto.

Cómo explicarlo oralmente

“Left-looking también puede paralelizarse: cada proceso forma contribuciones con las columnas anteriores y estas se combinan para calcular la columna actual. Right-looking calcula una columna y reparte las actualizaciones de la matriz restante. Implementé ambas con la misma distribución cíclica. En mis pruebas, right fue más rápido en todas las configuraciones; con n=1024 y tres procesos tardó 31,48 ms frente a 54,08 ms de left. El perfil muestra que left necesita más colectivas de reducción. Por eso elijo right para continuar, aclarando que es una conclusión sobre estas implementaciones.”

Qué sigue pendiente

Todavía medimos el kernel de factorización. El siguiente paso práctico es incluir distribución inicial y reunión final para comprobar si la ventaja se mantiene en el tiempo completo. Luego corresponde repetir dentro de un nodo y entre nodos del clúster.

Archivo en desde_cero	Contenido
src/cholesky_left_mpi.c	Left-looking distribuido y perfil de reducciones.
src/compare_mpi_algorithms.c	Comparación controlada, reunión y validación.
scripts/comparar_left_right_mpi.sh	Campaña reproducible.
scripts/analizar_left_right_mpi.py	Estadísticas, validación y gráficos.
resultados/left_right_mpi_20260922/	CSV, protocolo, perfil, huellas y fuentes.